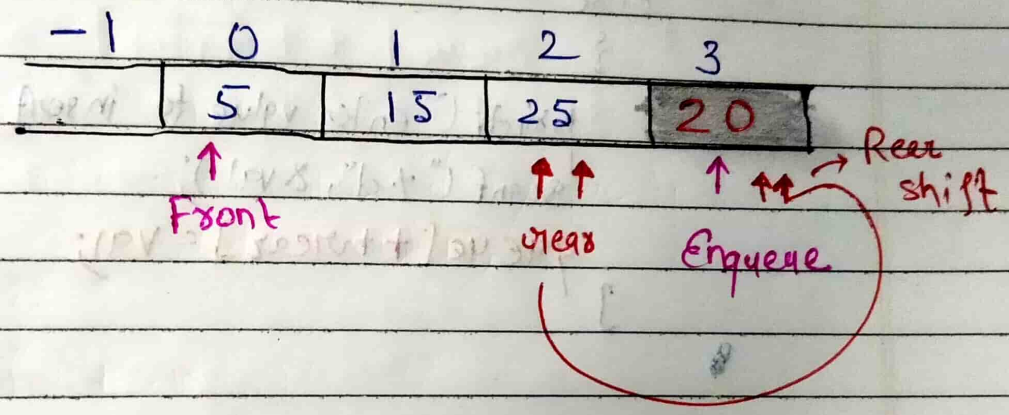
Enqueue :-

- Enqueue() is the function used to insert an element at the rear of the queue.

In FIFO (First In First Out), new elements always go to the rear.



Algorithm :

step 1: IF (rear == max-1)
print "queue is full"

step 2: queue[++rear] = VAL

step 3: Exit

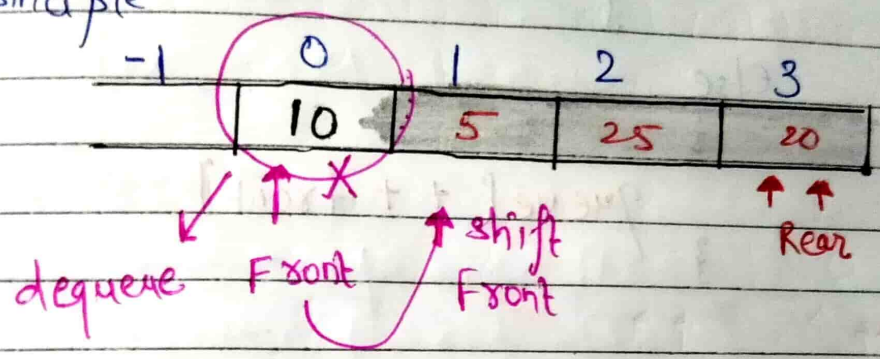
→ Pseudo code :-

```
int val,i;
if( rear == max-1)
  {
      printf("queue is full");
  }
else
  {
    printf("enter value to insert");
    scanf("%d", &val);
    queue[++rear] = val;
  }
```

Akash

→ dequeue :-

Dequeue() is the function used to remove an element from the front of the queue. following the FIFO (First In, First Out) principle.



Algorithm:-

Step 1:- IF (front == -1)
print empty queue

Step 2:- else,
queue[++front]

Step 3:- exit()

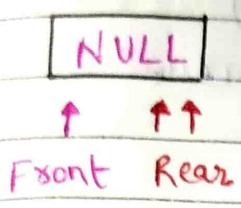
Akash

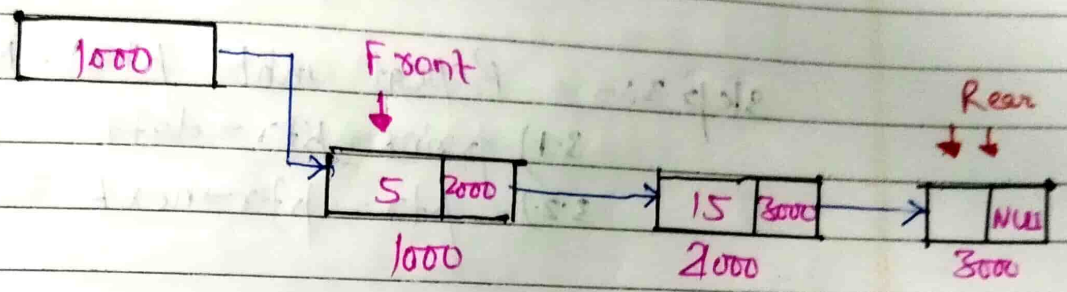
→ Pseudo code for dequeue:-

```
if (front == -1)
    {
        printf("empty queue");
    }
else
    {
        queue[ ++ front]
    }
```

Akash

Linked - list representation of Queue-





Algorithm to Create Linked List :-

step 1: Create nptr [new_node]

step 2: IF val = -1
goto step 6

step 3: IF (rear = = NULL)
front = rear = nptr

step 4: else,
4.1) ptr = front
4.2) Repeat until rear → next != NULL
4.3) rear = rear → next

step 5: rear → next = nptr.

step 6 : Exit

Akash

Algorithm to display:-

Step 1: IF (rear == NULL)
print "empty queue"

Step 2: else,
set ptr = front

Step 3:- Repeat until ptr != NULL
3.1) print ptr → data
3.2) ptr = ptr → next

Step 4: Exit()

Akash

→ C program to create & display linked list implementation of queue.

```
#include <stdio.h>
#include<stdlib.h>

struct node
  {
      int data;
      struct node *next;
  };

struct node *front = NULL, *rear = NULL

struct node *create_queue()
  {
      struct node *nptr;
      int val;
      printf("enter -1 to stop\n");
      scanf("%d",&val);
      while( val != -1)
        {
          nptr = (struct node *) malloc(sizeof(
            struct node));
          nptr->data = val;
          nptr->next = NULL;
        if( rear == NULL)
          {
              front = rear = nptr;
          }
```

Akash

```
else
{
    while( rear->next != NULL)
    {
        rear = rear->next;
    }
    rear->next = nptr;
}
printf("enter data\n");
scanf("%d",&val);
}
}

struct node *display()
{
    struct node *ptr;
    if( rear == NULL)
    {
        printf("empty queue");
    }
    else
    {
        ptr= front;
        while( ptr != NULL)
        {
            printf("%d->", ptr->data);
            ptr= ptr->data;
        }
    }
}
```

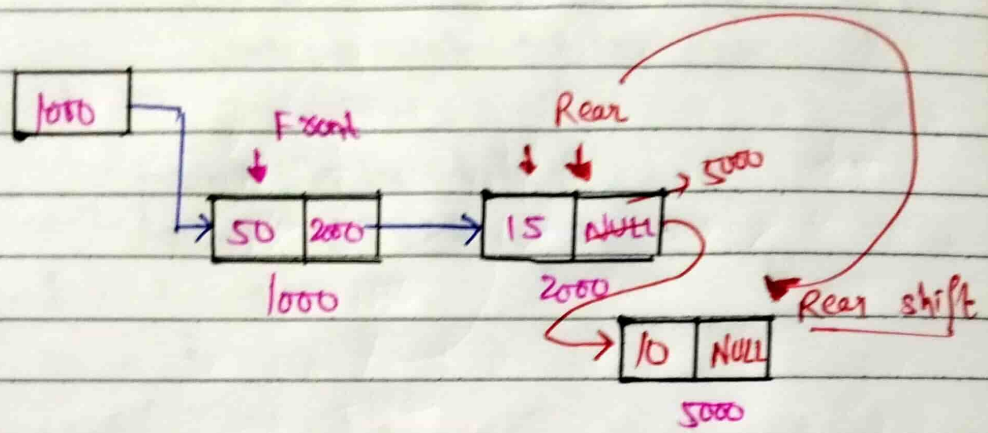
Akash

```
int main()
{
    create_queue();
    display();
    return 0;
}
```

Akash

→ Enqueue :- To insert a new element at the rear (end) of the queue

Same as in Linked List to insert a node at the end.



Algorithm :-

Step 1: Create new_node to insert [nptr]

Step 2: If front == NULL
2.1.) front = rear = nptr

Step 3: else,
3.1.) rear = front

Step 4: Repeat until rear → next != NULL
4.1.) rear = rear → next

Step 5.) rear → next = nptr

Step 6.) Exit

Akash

~~psu pseduo~~ pseudo

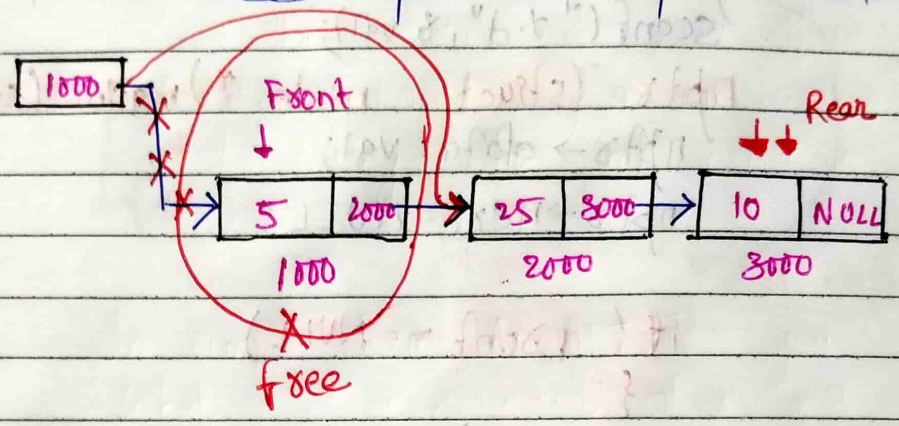
→ program to enqueue using linked list implementation in queue

```
struct node *nptr;
int val;
printf(" enter value to insert \n");
scanf("%d", &val);
nptr = (struct node *) malloc (sizeof (struct node));
nptr -> data = val;
nptr -> next = NULL;

if( front == NULL)
{
        front = rear = nptr;
}
else
{
    rear = front

while( rear -> next != NULL)
{
        rear = rear -> next;
}
    rear -> next = nptr;
}
```

Akash

→ dequeue → To remove the first element from the queue - following FIFO (First In, First Out)

Same as in Linked List to delete first node.



Algorithm:-

Step 1. IF front == NULL
print "empty list"

Step 2: else, ptr = front

Step 3: front = front → next;

Step 4: free (ptr);

Step 5: Exit

Akash

→ pseudo code for dequeue :-

```
struct node *ptr;
if( front == NULL)
{
    printf("empty list");
}
else
{
    ptr = front;
    front = front->next;
    free(ptr);
}
```

Akash

→ ADT (Abstraction Data Queue) Queue :-

ADT → An abstract data type (ADT) is mathematical model for a data type, which defines the behavior (operation) of the data structure and the rules for how those operation should function, without specifying the implementation details.

In simple terms, An ADT tells you what a data structure does, not how it does it.

A Queue is an abstraction data type that follows the Fast In First Out (FIFO) principle.

The main operation for the stacks are:

- Enqueue () → To insert a new element at the rear (end) of the queue.
- Dequeue () → To remove the first element from the queue following FIFO (First In First Out).

Akash

→ Implementation :-

1. Array
2. Linked List

} We have done this before.

Akash